SEMESTER PROJECT DOCUMENTATION

FocusTracker AI

A desktop productivity tracker for scheduled focus sessions, screenshot-based distraction detection, local model training, and productivity reports.

Course Teacher — **Ms Tayyaba Rasheed**

Submitted By:-

Hafiz Awais Zulfqar 1067	M Hassnain Pervez 1097
Aqeel Ahmad 1062	Abu Hurraira 1082
Saif Riaz 1109	

PROJECT TYPE
Electron desktop application

CORE STACK
React, Vite, Electron, Prisma, PostgreSQL

ML SCOPE
Dataset preprocessing, training, analysis, evaluation

CURRENT MODEL
Local logistic regression model trained on real screenshots

Repository: sp-focus-tracker

May 2026

Documentation Map

This report is organized as a 15-page technical document. It covers the app structure, frontend, Electron backend, database relations, ML pipeline, trained model outputs, setup, and testing flow.

The document is built with fixed A4 page sections to prevent the broken card and table splitting shown in the previous PDF.

Project Overview

FocusTracker AI helps a user plan focus tasks, run timed sessions, capture screenshots during the session, classify screenshots as focused or distracted, and review historical performance.

64
Real screenshots in current dataset

34
Focused training images

30
Distracted training images

Main Objectives

Planning
Create tasks with task name, scheduled time, and focus duration.

Focus Mode
Start a session from an alarm or dashboard action and keep a strict timer visible.

Evidence Capture
Take screenshots during the focus session using the Electron desktop layer.

Local Analysis
Run a trained local model to classify focused and distracted screen activity.

Reports
Store session summaries and show focus trends, best score, and recent history.

Academic ML
Provide a complete local ML workflow: preprocess, train, evaluate, analyze.

Scope

Area	Included In This Project
Desktop app	Electron shell, secure preload bridge, React renderer, local app flow.
Productivity	Todo scheduling, alarms, focus sessions, score summaries, reports.
AI / ML	Local image feature extraction and binary focused/distracted classifier.
Data	PostgreSQL database through Prisma models for users, todos, and sessions.

Technology Stack

The project uses a desktop-first stack. React handles the interface, Electron handles desktop access, Prisma handles database access, and the local ML folder handles model training and prediction.

Layer	Technology	Purpose
Desktop shell	Electron	Runs the app as a native desktop application and provides main-process APIs.
Frontend	React	Builds the Auth, Dashboard, Focus Session, and Reports screens.
Build tool	Vite	Runs fast local development and creates frontend build output.
UI motion	Framer Motion	Provides transitions and page/component animation.
Charts	Recharts	Displays focus trends in the reports screen.
Database ORM	Prisma	Maps JavaScript service calls to PostgreSQL models.
Database	PostgreSQL	Stores users, tasks, and focus session results.
Security	bcryptjs, keytar	Hashes passwords and stores the current user ID securely.
Scheduling	node-schedule	Schedules task alarms from todo data.
Images	sharp	Compresses screenshots and extracts ML image features.
Cloud-ready storage	AWS S3/R2 SDK	Provides object storage support through the R2 service file.
Local ML	Node.js scripts	Preprocesses screenshots, trains the model, evaluates it, and predicts one image.

Main app command

`npm run dev`

Runs Vite and Electron together for development.

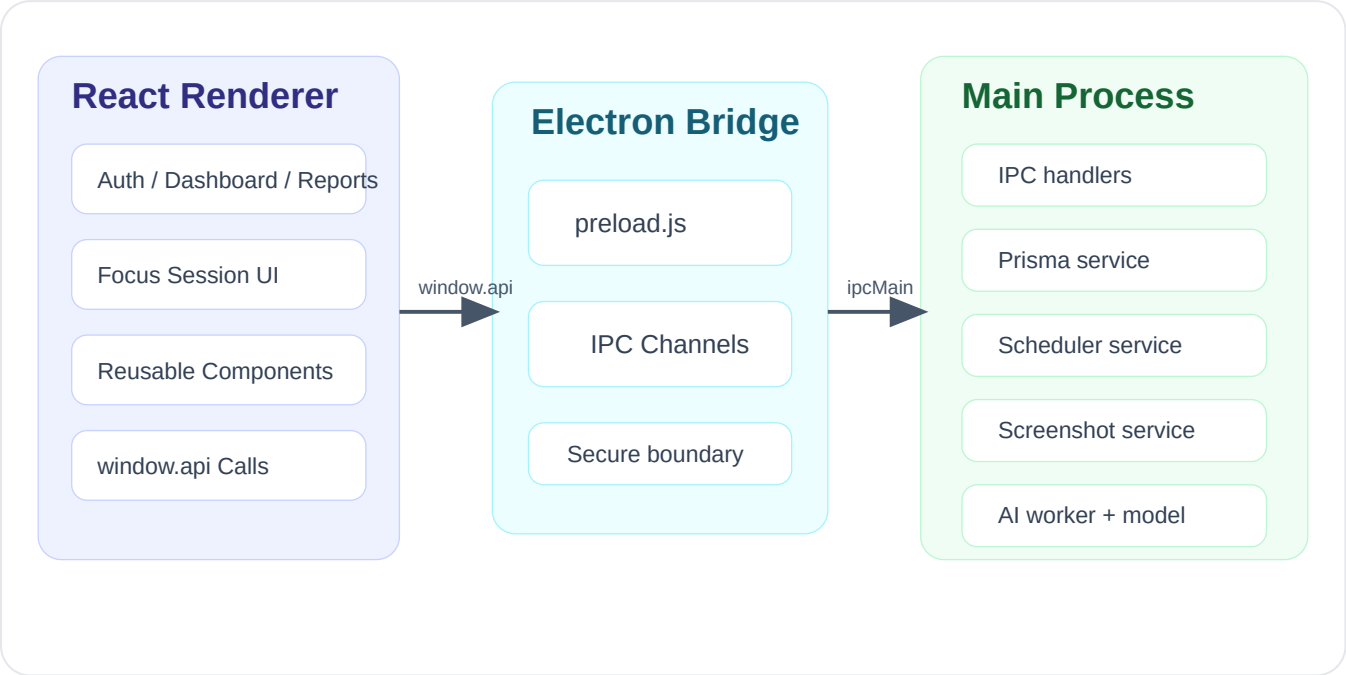
Model command

`npm run ml:all`

Runs preprocessing, training, and evaluation.

System Architecture

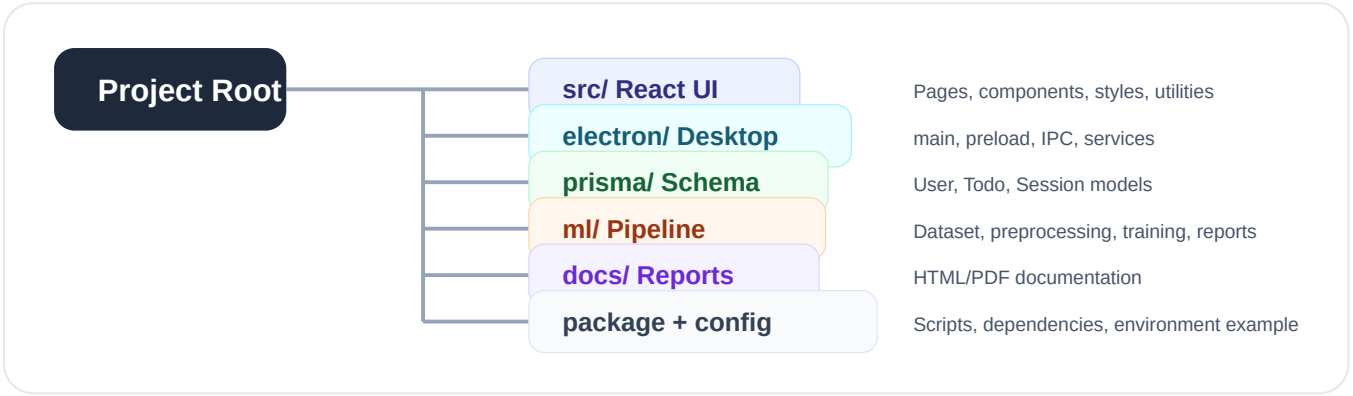
The app has two major runtime areas: the React renderer process and the Electron main process. The preload file exposes a controlled API between them.



Area	Main Files	Role
Renderer	`src/main.jsx`, `src/components/App/App.jsx`	Loads the React app and controls screen-level navigation.
Bridge	`electron/preload.js`	Exposes safe methods to the renderer.
Main process	`electron/main.js`	Creates the window, registers IPC, and controls app-level behavior.

Folder Structure

The project is separated into frontend source, Electron backend files, Prisma database schema, ML pipeline, documentation, build output, and raw screenshot data.



Source Structure

```
src/
  main.jsx
  pages/
    Auth/
    Dashboard/
    FocusSession/
    Reports/
  components/
    App/
    Sidebar/
    TimerRing/
    TodoCard/
    SessionCard/
    ResultScreen/
    FocusChart/
    icons/
  lib/
  utils/
  constants/
  styles/
```

Backend And ML Structure

```
electron/
  main.js
  preload.js
  ipc/
    auth.js
    todos.js
    session.js
    reports.js
  services/
    prisma.js
    scheduler.js
    screenshot.js
    ai-worker.js
    r2.js

ml/
  dataset/raw/
  processed/
  models/
  reports/
  lib/
  preprocess.js
  train.js
  evaluate.js
  analyze.js
```

Frontend Structure

The frontend is a React renderer app. The main app controller decides whether to show authentication, dashboard, focus session, or reports.

Frontend Area	Files	Purpose
App shell	`src/main.jsx`, `src/components/App/App.jsx`	Mounts React and controls the high-level screen state.
Authentication	`src/pages/Auth/Auth.jsx`	Signup and login screen.
Dashboard	`src/pages/Dashboard/Dashboard.jsx`	Task creation, todo list, alarm modal, session start.
Focus session	`src/pages/FocusSession/FocusSession.jsx`	Active timer, progress ring, session completion UI.
Reports	`src/pages/Reports/Reports.jsx`	Historical focus score summaries and charts.

Reusable Components

Navigation

`Sidebar`
`NavItem`

Focus UI

`TimerRing`
`ResultScreen`
`AlarmModal`

Cards

`TodoCard`
`SessionCard`
`StatCard`

Inputs

`Field`
`InputField`
`StyledInput`

Charts

`FocusChart`
`StatTile`
`StatPill`

Visual System

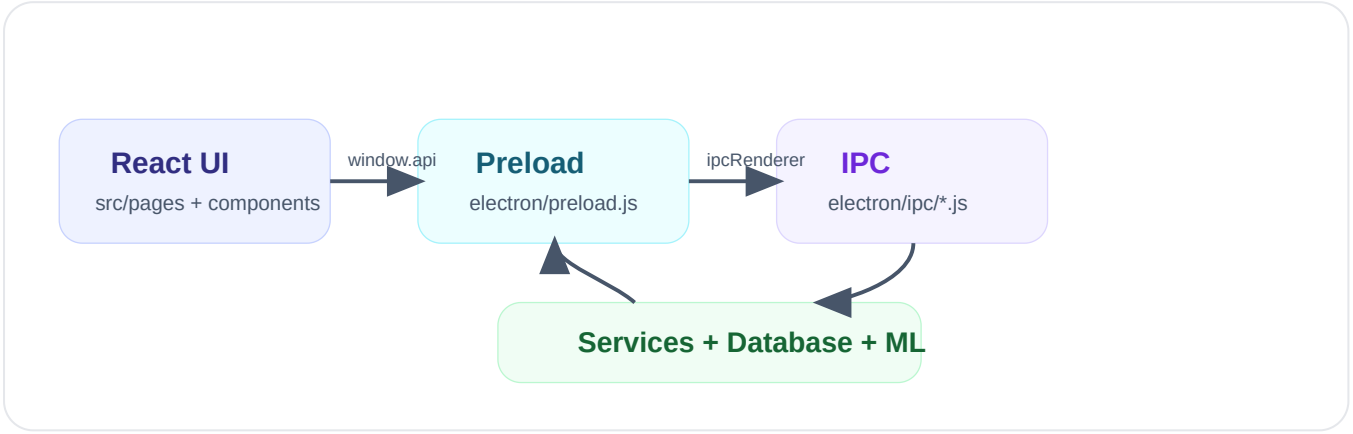
`Background`
`Particle`
`icons/`

Frontend Reading Order

1. Read `src/main.jsx`.
2. Read `src/components/App/App.jsx`.
3. Read pages in this order: Auth, Dashboard, FocusSession, Reports.
4. Read shared helpers: `src/lib/api.js`, `src/utils/time.js`, `src/utils/score.js`.
5. Read reusable components only after you understand the pages.

Electron And IPC Structure

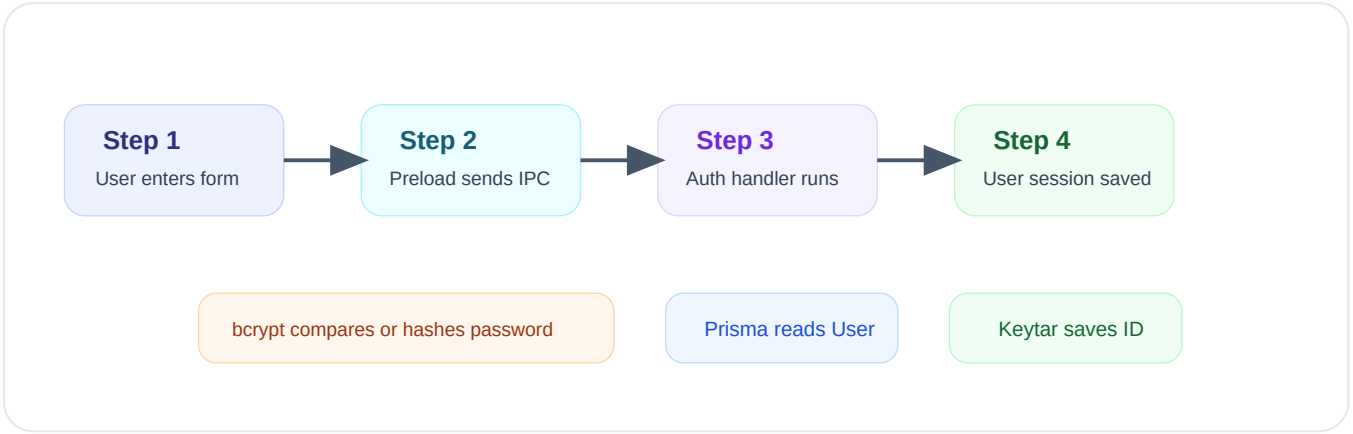
Electron owns the desktop features that the browser cannot safely access directly. React calls exposed preload methods, preload sends IPC messages, and IPC handlers use backend services.



File	What To Understand
`electron/main.js`	App window creation, preload loading, IPC registration, session close protection.
`electron/preload.js`	The public API exposed to the React renderer.
`electron/ipc/auth.js`	Signup, login, logout, saved current user lookup.
`electron/ipc/todos.js`	Todo create, read, update, delete, scheduling integration.
`electron/ipc/session.js`	Focus session creation, timer, screenshot capture, completion.
`electron/ipc/reports.js`	Report data retrieval and summary preparation.

Authentication Flow

Authentication uses React input forms, IPC calls, bcrypt password hashing, Prisma database access, and Keytar for saved session identity.



Signup path

User data goes from `Auth.jsx` to `auth:signup`. Password is hashed, user is created, and current user ID is saved.

Login path

Email lookup runs through Prisma, password hash is checked with bcrypt, and the saved user ID opens the dashboard.

Startup path

`App.jsx` calls `api.getUser()`. If a saved user exists, Auth is skipped.

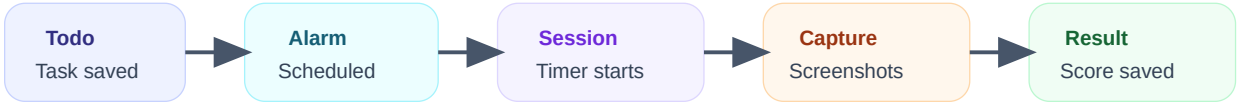
Logout path

The current user ID is cleared from Keytar and the app returns to Auth.

> The saved session is not fake login bypass. It is a restored local session from Keytar.

Tasks, Alarms, And Focus Sessions

The normal app workflow starts from a todo task, schedules an alarm, starts a focus session, captures screenshots, analyzes them, and saves the final result.

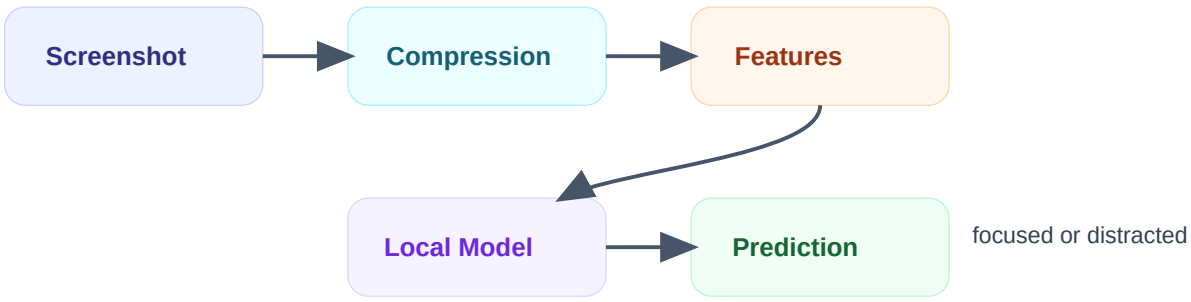


During an active session, Electron prevents accidental window close and waits for timer completion.

Step	Main Files	Outcome
Create task	`Dashboard.jsx`, `electron/ipc/todos.js`	Todo row is saved with task name, scheduled time, duration.
Schedule alarm	`electron/services/scheduler.js`	Task alarm is registered and later emitted to the renderer.
Start session	`electron/ipc/session.js`	Session row is created and active timer begins.
Capture screen	`electron/services/screenshot.js`	Screenshots are captured and optimized.
Complete session	`electron/services/ai-worker.js`	Model output is converted into score, summary, and distraction details.

Screenshot And Local Model Analysis

The local analyzer mode uses the trained model file in `ml/models/focus-logreg.json`. Screenshots become image features, features become probabilities, and probabilities become focused or distracted labels.



Analyzer switch

`FOCUS_ANALYZER=local`

Enables the local trained model in the app.

Model file

`ml/models/focus-logreg.json`

Created by the training script.

Feature source

`ml/lib/common.js`

Extracts color, brightness, saturation, edge, grid, and histogram features.

Runtime worker

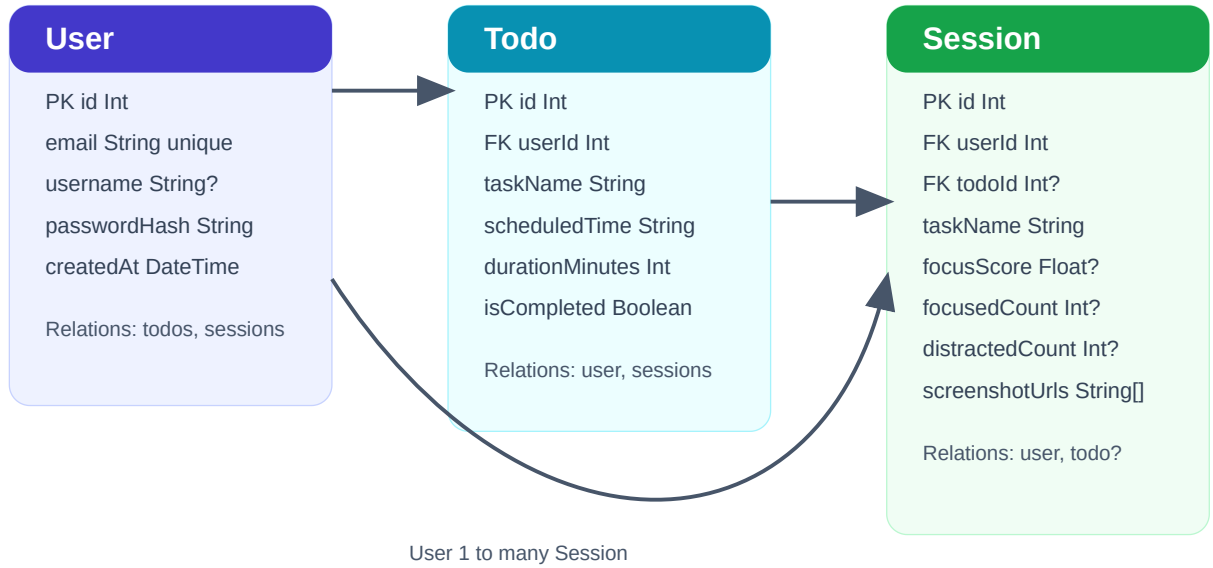
`electron/services/ai-worker.js`

Calls predictor logic during session completion.

```
{
  "focusScore": 80,
  "total": 10,
  "focused": 8,
  "distracted": 2,
  "summary": "Local trained model analyzed 10 screenshots."
}
```

Database ER Diagram

The Prisma schema contains three main models: User, Todo, and Session. The ER diagram below is included directly in the PDF so the database relations are visible without opening the code.



Database Models And Relations

This page gives the field-level database documentation for the current Prisma schema.

Model	Field	Type	Purpose
User	`id`	Int	Primary key.
User	`email`	String unique	Login identifier.
User	`username`	String?	Optional display name.
User	`passwordHash`	String	Hashed password from bcrypt.
Todo	`userId`	Int	Foreign key to User.
Todo	`taskName`	String	Name of planned focus task.
Todo	`scheduledTime`	String	Time used by scheduler.
Todo	`durationMinutes`	Int	Focus session length.
Session	`userId`	Int	Foreign key to User.
Session	`todoId`	Int?	Optional link to Todo. Uses SetNull on delete.
Session	`focusScore`	Float?	Final score after screenshot analysis.
Session	`focusedCount`	Int?	Number of focused screenshots.
Session	`distractedCount`	Int?	Number of distracted screenshots.
Session	`screenshotUrls`	String[]	Saved screenshot paths or URLs.

Relationship Summary

User to Todo

One user can own many planned tasks.

User to Session

One user can have many completed or active focus sessions.

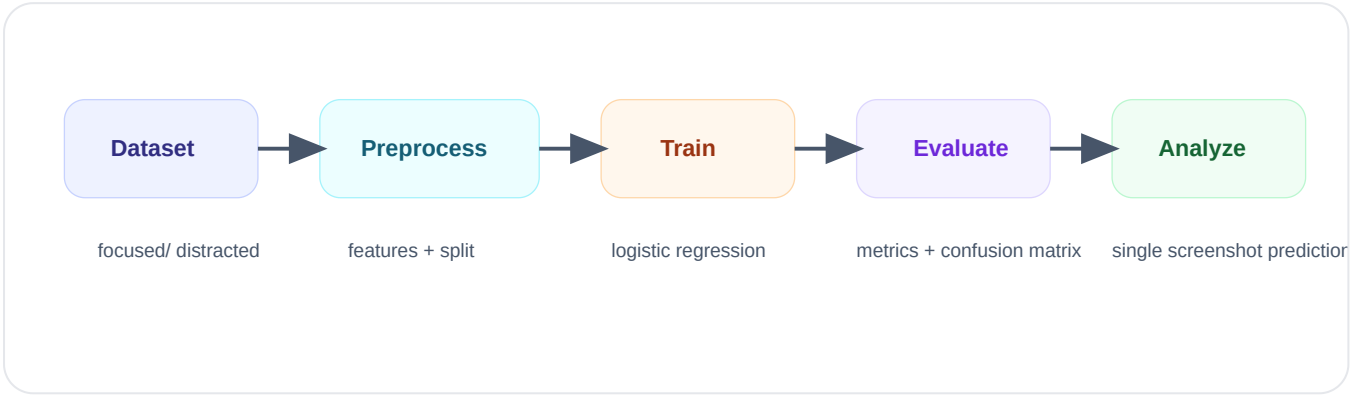
Todo to Session

A session may belong to a todo. If the todo is deleted, session history remains.

```
model Session {
  user User @relation(fields: [userId], references: [id])
  todo Todo? @relation(fields: [todoId], references: [id], onDelete: SetNull)
}
```

ML Pipeline And Current Model

The ML folder provides a complete local training workflow. The current fake/demo data was removed and the model was retrained using 64 real labeled screenshots.



51
Training records

13
Evaluation records

100%
Current test accuracy

Requirement	File	Output
Dataset preprocessing	`ml/preprocess.js`	`ml/processed/dataset.json`, train/test JSON.
Model training	`ml/train.js`	`ml/models/focus-logreg.json`, training report.
Evaluation	`ml/evaluate.js`	Accuracy, precision, recall, F1, confusion matrix.
Single image analysis	`ml/analyze.js`	Prediction for one screenshot.
Live prediction	`ml/lib/predictor.js`	Reusable predictor for Electron worker.

> The current 100% evaluation result is from a small 13-image test split. It is useful for project demonstration, but a larger dataset is needed for stronger generalization.

Setup, Testing, And Future Work

This final page gives the commands needed to run the app, train the model, and verify the local analyzer.

Environment

```
DATABASE_URL=postgresql://user:pass@localhost:5432/focus-tracker
FOCUS_ANALYZER=local
LOCAL_MODEL_PATH=
```

App Commands

```
npm install
npm run prisma:generate
npm run prisma:push
npm run dev
```

ML Commands

```
npm run ml:preprocess
npm run ml:train
npm run ml:evaluate
npm run ml:analyze -- ml/dataset/raw/focused/focused_real_001.jpeg
npm run ml:all
```

Manual Testing Checklist

Test	Expected Result
Open a study PDF, report page, code file, or assignment page	Model should classify it as focused.
Open Instagram, Facebook, YouTube, Netflix, game, or sports feed	Model should classify it as distracted.
Create a todo with scheduled time	Alarm should trigger and show the focus start modal.
Complete a focus session	Session score, focused count, distracted count, and summary should be saved.
Open Reports	Recent sessions and focus chart should be visible.

Future Work

- Collect a larger balanced dataset for focused and distracted screens.
- Add automated tests for IPC handlers, scheduler behavior, and ML scripts.
- Add screenshot privacy controls and optional screenshot deletion policies.
- Add stronger validation for task duration, scheduled time, and active session state.
- Improve the local model with more robust computer-vision features or a lightweight neural model.

> Primary reading companion: `docs/CODE_READING_ORDER.md`. Use it when you want to understand the files in sequence.